

LiamConnolly_AO2_CodingBasics.Rmd

Liam Connolly

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file <FirstLast>_AO2_CodingBasics.Rmd (replacing <FirstLast> with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.

```
#sequence
Liam_sequence <- seq(from = 1, to = 55, by = 5)
print(Liam_sequence)
#starts at 1 hence the from, to signifies the ending number, and by is the count of increase
2. Compute the mean and median of this sequence.
mean(Liam_sequence)
#mean is 26
median(Liam_sequence)
#median is 26 as well
3. Ask R to determine whether the mean is greater than the median.
mean(Liam_sequence) > median(Liam_sequence)
4. Insert comments in your code to describe what you are doing.
```

```
#1. starts at 1 hence the from, to signifies the ending number, and by is the count of increase
Liam_sequence <- seq(from = 1, to = 55, by = 5)
print(Liam_sequence)
```

```
## [1] 1 6 11 16 21 26 31 36 41 46 51
```

```
#2. just expressing mean and median as factor of my sequence name
mean(Liam_sequence)
```

```
## [1] 26
```

```
median(Liam_sequence)
```

```
## [1] 26
```

#3. I just compared the given mean and median of my sequence, I couldn't figure out how to use the if/e

```
mean(Liam_sequence) > median(Liam_sequence)
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
names_student <- c("Luke", "Joe", "Julia", "Steve") #vector is string or is it character???
test_scores <- c(50, 90, 84, 70) #vector is numeric
scholarship_test <- c(TRUE, TRUE, TRUE, FALSE) #vector is logical
print(names_student)
```

```
## [1] "Luke" "Joe" "Julia" "Steve"
```

```
print(test_scores)
```

```
## [1] 50 90 84 70
```

```
print(scholarship_test)
```

```
## [1] TRUE TRUE TRUE FALSE
```

```
student_progress <- data.frame("StudentName" = names_student, "TestScore" = test_scores, "OnScholarship"
print(student_progress)
```

```
##   StudentName TestScore OnScholarship
## 1      Luke        50          TRUE
## 2       Joe        90          TRUE
## 3     Julia        84          TRUE
## 4     Steve        70          FALSE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: not sure if this is the answer but is it because this dataframe effectively evaluates real world data and uses mutiple types of data ie. logical, string/character, and numeric whereas a matrix is more for mathematical equations that are all numeric

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else, #this is my take on it, not sure if I did it right...??? I watc
#pass_fail_check <- function(score) {if (score > 50){print("Pass")} else {print("Fail")}}
#pass_fail_check(65)
#pass_fail_check(40)
#11. Create a function using ifelse()
pass_fail_check2 <- function(score) {result <- ifelse(score > 50, "Pass", "Fail")
print(result)
}
#okay...so I think I did this okay, maybe...I kept running into error codes with the brackets until I m
pass_fail_check2(40)
```

```
## [1] "Fail"
```

```
pass_fail_check2(60)
```

```
## [1] "Pass"
```

```
#12a. Run the first function with the value 52.5, again made me put a # to knit it
#pass_fail_check(52.5)
#came out as pass
#12b. Run the second function with the value 52.5
pass_fail_check2(52.5)
```

```
## [1] "Pass"
```

```
#came out as pass
#13a. Run the first function with the vector of test scores
#test_scores <- c(50, 90, 84, 70)
#pass_fail_check(test_scores)
# it freaks out on me, likely because I don't have the if/else I'm guessing..., also made me put a # to
#13b. Run the second function with the vector of test scores
pass_fail_check2 <- function(score) {result <- ifelse(score > 50, "Pass", "Fail")
print(result)
}
test_scores <- c(50, 90, 84, 70)
pass_fail_check2(test_scores)
```

```
## [1] "Fail" "Pass" "Pass" "Pass"
```

#coincidentally had a score that was exactly 50 so made for interesting false

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer:so I’m guessing that `if...else` is supposed to be in the first section, and I didn’t have that as I kept getting error codes, so I’m kind of flying blind here. I need to submit the assignment but I’ll upload an updated one when I figure out the `if...else` in the first section...anyways. `Ifelse` is the one that worked and my immediate guess why that was is that `if...else` is designed for a single true or false question, a vector breaks it as its more than one value. Whereas, `ifelse` works within a function and can check multiple values and provide answers hence why it didn’t break.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)